Sinatra com Ruby

Da primeira rota a uma aplicação web.

Fernando Buligon

O que vamos aprender

Entender

O papel do Sinatra no desenvolvimento web com Ruby.

Ler e escrever

Rotas, parâmetros, respostas e código reutilizável.

Reconhecer na prática

Esses recursos na aplicação de tarefas desenvolvida.

Pouca estrutura para começar

Sinatra é uma DSL para criar aplicações web em Ruby: uma linguagem de domínio específico expressa no próprio Ruby.

Sua interface aproxima o código da requisição HTTP: escolher uma rota e executar um bloco.

Uma rota é código Ruby

```
get '/ola' do
  'Olá, Sinatra!'
end
```

get

Método oferecido pelo Sinatra para registrar uma rota.

do ... end

Bloco Ruby executado quando a rota é atendida.

Última expressão

Aqui, a string se torna o corpo da resposta.

Exemplo didático independente do projeto.

Do arquivo ao navegador

```ruby
# ola.rb
require 'sinatra'

get '/' do
  'Olá, Sinatra!'
end
```

```
gem install sinatra rackup puma
ruby ola.rb
```

Abra **http://localhost:4567**.

Exemplo mínimo. No projeto, usamos Bundler para gerenciar as dependências.

Onde o Sinatra entra

Cliente
Envia HTTP

→

Puma + Rack
Servidor e interface

→

Sinatra
Seleciona a rota

→

Bloco Ruby
Produz a resposta

O cliente recebe status, cabeçalhos e corpo da resposta.

O código da aplicação decide o que fazer dentro de cada rota.

Método + caminho → ação

```
get '/artigos' do
  'Lista de artigos'
end

post '/artigos' do
  status 201
  'Artigo criado'
end
```

A mesma URL pode ter comportamentos diferentes conforme o método HTTP.

O Sinatra também oferece **put**, **patch** e **delete**.

Exemplo didático: nenhuma gravação no banco é realizada aqui.

Como os dados chegam

```
get '/artigos/:id' do
  id = params['id']
  filtro = params['busca']
  "Artigo #{id}; busca: #{filtro}"
end
```

Na URL

/artigos/42?busca=ruby → id: "42", busca: "ruby".

No corpo JSON

No projeto: JSON.parse(request.body.read).

Na aplicação

Validar e converter os valores recebidos.

O corpo JSON é lido explicitamente; não vira params automaticamente.

Controle da resposta HTTP

```
require 'json'

get '/saude' do
  content_type :json
  status 200
  { status: 'ok' }.to_json
end
```

content_type

Define o tipo do conteúdo enviado.

status

Define o código HTTP da resposta.

to_json

Serializa os dados com a biblioteca JSON do Ruby.

Filtros e helpers

```
before do
  content_type :json
end

helpers do
  def invalid_input(message)
    halt 400, { error: message }.to_json
  end
end
```

before

Executa lógica antes das rotas. No projeto, define JSON como padrão.

helpers

Agrupa métodos usados nos blocos das rotas.

halt

Interrompe o fluxo e devolve uma resposta.

Trecho real de app.rb; a biblioteca JSON é carregada no início do arquivo.

Também serve HTML

```
get '/' do
  content_type :html
  erb :index
end
```

views/index.erb

Template renderizado pela rota inicial.

public/

Arquivos estáticos: CSS e JavaScript.

Uma aplicação

Pode oferecer páginas HTML e respostas JSON.

Trecho real de app.rb. A rota sobrescreve o tipo JSON definido no filtro.

Liberdade pede organização

Ponto forte

Uma rota pode ser apresentada e compreendida em poucas linhas.

Responsabilidade

O projeto precisa definir suas validações, persistência e organização.

Ao crescer

Separe responsabilidades; Sinatra também permite aplicações com Sinatra::Base.

Neste seminário, o estilo clássico em app.rb facilita acompanhar o fluxo.

Os conceitos na aplicação

Sinatra
Rotas, filtro before, helpers e renderização ERB.

SQLite
Persistência escolhida para este exemplo.

Ruby + JSON
Regras, validações e serialização das respostas.

HTML + CSS + JavaScript
Interface que faz requisições às rotas.

A aplicação de tarefas serve como laboratório para observar a ferramenta.

Uma rota real, passo a passo

```
post '/tasks' do
  attributes = json_attributes(['title'])
  task = create_task(attributes['title'])
  status 201
  task.to_json
end
```

1 · Receber e validar

json_attributes é um helper criado no projeto.

2 · Persistir

create_task usa SQLite; não é um recurso nativo do Sinatra.

3 · Responder

Sinatra define o status; Ruby serializa a tarefa.

Trecho real de app.rb. A regra da aplicação fica dentro do fluxo da rota.

Do código ao comportamento

01 · Iniciar

bundle exec ruby app.rb — abrir http://127.0.0.1:4567.

02 · Mostrar HTML e JSON

Comparar / com /tasks e localizar os dois blocos get em app.rb.

03 · Criar uma tarefa

Relacionar a ação da interface ao bloco post e à resposta 201.

04 · Observar uma falha

No Postman, enviar título vazio e explicar helper, halt e status 400.

A demonstração destaca os recursos do Sinatra presentes em cada ação.

Da requisição ao bloco Ruby

Sinatra oferece uma forma direta de definir o comportamento de uma aplicação web.

Rotas conectam HTTP ao código. Filtros, helpers e templates ajudam a construir a aplicação.

A API de tarefas é um exemplo de como combinar esses recursos.

Para consultar e experimentar

Documentação oficial

sinatrarb.com/intro.html

Rotas, filtros, helpers, templates e aplicações modulares.

Código do seminário

github.com/FernandoBuligon/seminario-tds

app.rb, Gemfile e README.md.

Recorte utilizado

Projeto com Sinatra ~> 4.2 e Ruby ≥ 3.2; dependências no Gemfile.lock.